

# Embedded Rust

# Кто мы такие

**Михаил Чешков**

Разрабатываю на JS  
и немного на C++

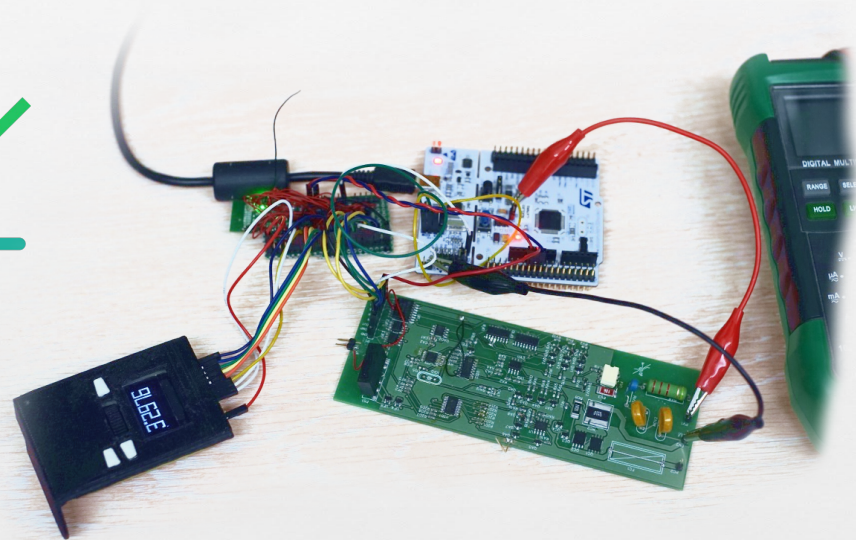
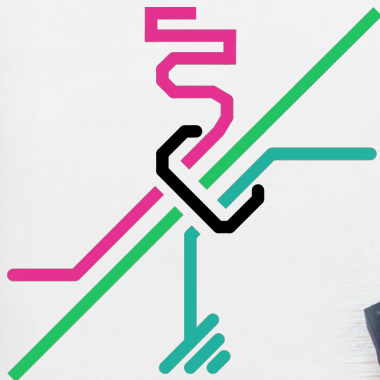
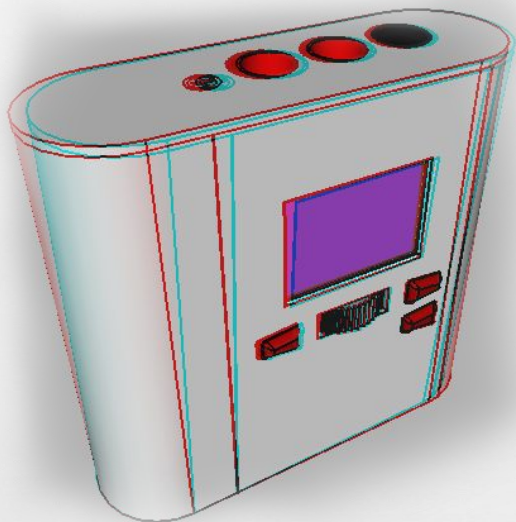


**Андрей Строков**

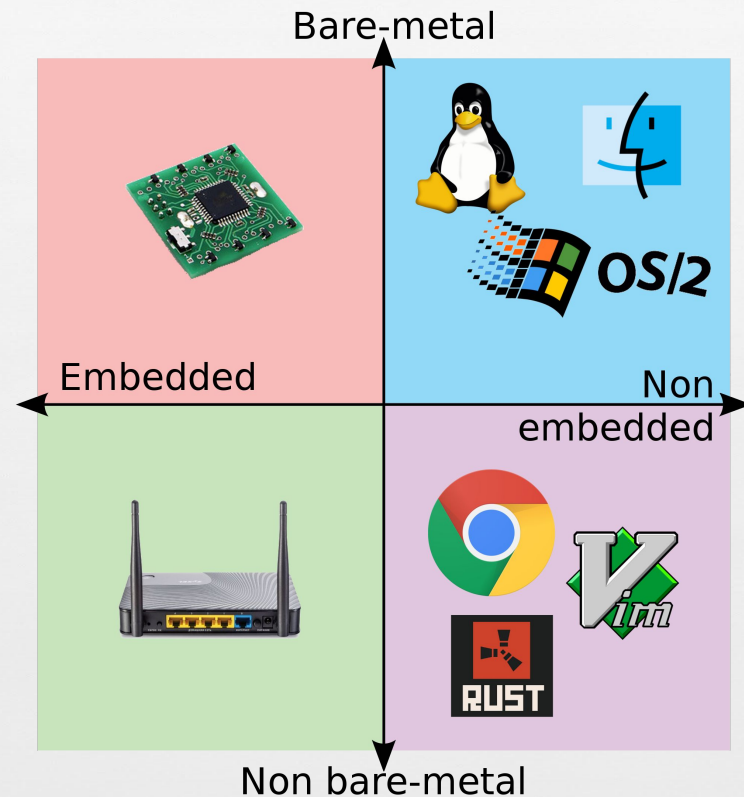
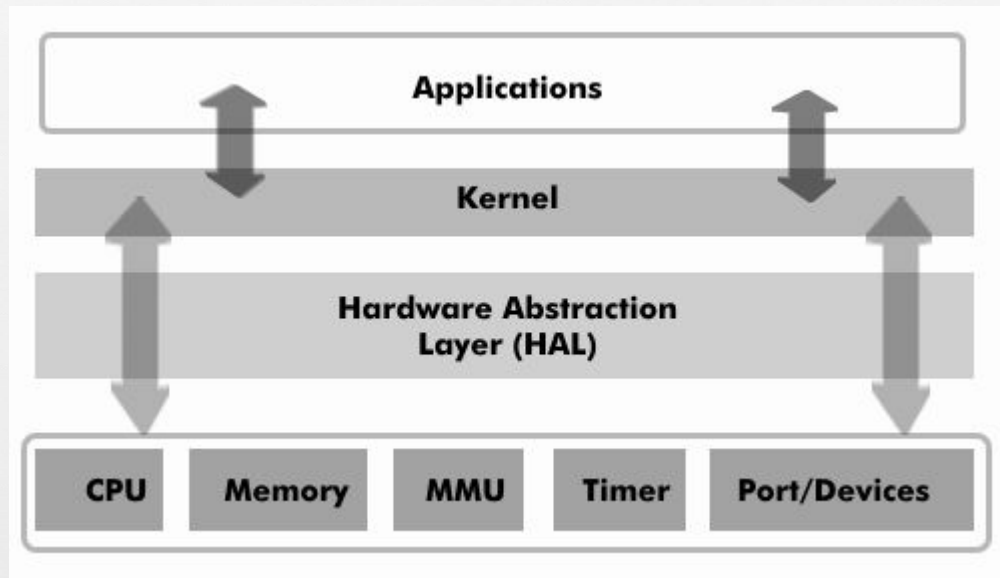
Разрабатываю электронику  
пишу прошивки



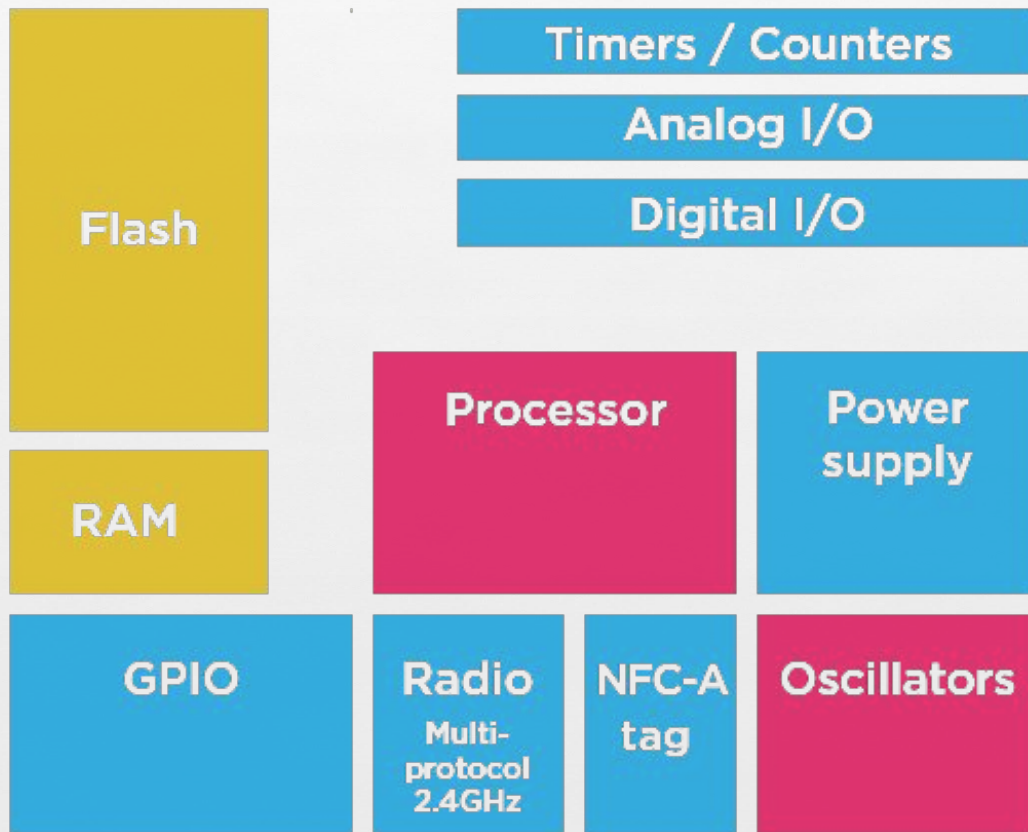
# Kviver team и мультиметр



# Что такое embedded



# Про микроконтроллеры



# Про микроконтроллеры

- Memory-mapped регистры
- Прерывания
- DMA
- Статически слинкованная прошивка
- Фиксированный адрес точки входа
- Обычно пишется на C/C++, иногда на ASM



существование include guard, необходимость писать имя функции в 2 (3) местах и т.п.

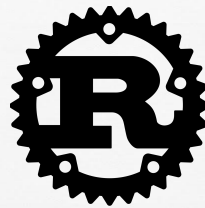
Конфигурация дефайнами

Триграфы!

Легко стрелять в ноги

Сложно собирать — надо самому писать Makefile и собирать франкештейна

принципиально новая<sup>©™</sup>  
модель владения памятью



стандартная система  
модулей и сборки

нормальные макросы

типизация 👍



**PROBLEM?**



**уицид**

**с**

**++**

# Как написать прошивку за 60 секунд

- Makefile
- linker script
- startup (ASM)
- source ©
- компилятор
- программатор

# Как написать прошивку за 60 секунд

```
1  /* Entry Point */
2  ENTRY(Reset_Handler)
3
4  /* Highest address of the user mode stack */
5  _estack = 0x20018000; /* end of RAM */
6
7  /* Specify the memory areas */
8  MEMORY {
9      RAM (xrw)  : ORIGIN = 0x20000000, LENGTH = 96K
10     FLASH (rx) : ORIGIN = 0x80000000, LENGTH = 1024K
11 }
12
13 SECTIONS {
14     /* The startup code goes first into FLASH */
15     .isr_vector :
16     {
17         . = ALIGN(8);
18         KEEP(*(.isr_vector)) /* Startup code */
19         . = ALIGN(8);
20     } >FLASH
```

# Как написать прошивку за 60 секунд

```
22 .text :
23 {
24     . = ALIGN(8);
25     /* .text sections (code) */
26     *(.text)
27     . = ALIGN(8);
28 } >FLASH
29
30 .rodata :
31 {
32     . = ALIGN(8);
33     /* constants, strings, etc. */
34     *(.rodata)
35     . = ALIGN(8);
36 } >FLASH
37
```

```
/* Initialized data sections goes into RAM,
load LMA copy after code */
.data : {
    . = ALIGN(8);
    _sdata = .;
    *(.data)
    . = ALIGN(8);
    _edata = .;
} >RAM AT> FLASH

/* Uninitialized data section */
. = ALIGN(4);
.bss : {
    /* define a global symbol at bss start */
    _sbss = .;
    __bss_start__ = _sbss;
    *(.bss)
    . = ALIGN(4);
    /* define a global symbol at bss end */
    _ebss = .;
    __bss_end__ = _ebss;
} >RAM
```

# Как написать прошивку за 60 секунд

```
71 .section .isr_vector,"a",%progbits
72 .type g_pfnVectors, %object
73 .size g_pfnVectors, .-g_pfnVectors
74
75
76 g_pfnVectors:
77 .word _estack
78 .word Reset_Handler
79 .word NMI_Handler
80 .word HardFault_Handler
81 .word MemManage_Handler
82 .word BusFault_Handler
83 .word UsageFault_Handler
84 .word 0
85 .word 0
86 .word 0
87 .word 0
88 .word SVC_Handler
89 .word DebugMon_Handler
90 .word 0
```

```
27 Reset_Handler:
28     ldr sp, =_estack
29     movs r1, #0
30     b LoopCopyDataInit
31
32 CopyDataInit:
33     ldr r3, =_sidata
34     ldr r3, [r3, r1]
35     str r3, [r0, r1]
36     adds r1, r1, #4
37
38 LoopCopyDataInit:
39     ldr r0, =_sdata
40     ldr r3, =_edata
41     adds r2, r0, r1
42     cmp r2, r3
43     bcc CopyDataInit
44     ldr r2, =_sbss
45     b LoopFillZerobss
46
```

```
46
47 /* Zero fill the bss segment. */
48 FillZerobss:
49     movs r3, #0
50     str r3, [r2], #4
51
52 LoopFillZerobss:
53     ldr r3, =_ebss
54     cmp r2, r3
55     bcc FillZerobss
56     bl main
57
```

# Как написать прошивку за 60 секунд

```
17 int main(void) {
18     SET_BIT(RCC->APB2ENR, LL_APB2_GRP1_PERIPH_SYSCFG);
19
20     SET_BIT(RCC->CR, RCC_CR_HSION);
21     while(READ_BIT(RCC->CR, RCC_CR_HSIRDY) != (RCC_CR_HSIRDY)){
22
23     MODIFY_REG(RCC->CFGR, RCC_CFGR_SW, LL_RCC_SYS_CLKSOURCE_HSI);
24     while(READ_BIT(RCC->CFGR, RCC_CFGR_SWS) != LL_RCC_SYS_CLKSOURCE_STATUS_HSI) {}
25
26     SET_BIT(RCC->AHB2ENR, LL_AHB2_GRP1_PERIPH_GPIOA);
27
28     MODIFY_REG(
29         LD2_GPIO_Port->MODER,
30         (GPIO_MODER_MODE0 << (POSITION_VAL(LD2_Pin) * 2U)),
31         (LL_GPIO_MODE_OUTPUT << (POSITION_VAL(LD2_Pin) * 2U))
32     );
33
34     while (1) {
35         WRITE_REG(LD2_GPIO_Port->BSRR, LD2_Pin);
36         for(uint32_t i = 0; i < 1000000; i++) {}
37         WRITE_REG(LD2_GPIO_Port->BRR, LD2_Pin);
38         for(uint32_t i = 0; i < 1000000; i++) {}
39     }
40 }
```

# Как написать прошивку за 60 секунд

```
19 MCU = -mcpu=cortex-m4 -mthumb -mfpv4-sp-d16 -mfloat-abi=hard
20
21 ASFLAGS = $(MCU) -Og -Wall -fdata-sections -ffunction-sections
22 CFLAGS = $(MCU) $(C_DEFS) $(C_INCLUDES) -Og -Wall -fdata-sections -ffunction-sections -g -gdwarf-2
23 LDFLAGS = $(MCU) -specs=nano.specs -TSTM32L476RGTx_FLASH.ld -lc -lm -lnosys -Wl,--gc-sections
24
25 # default action: build all
26 all: $(BUILD_DIR)/$(TARGET).elf $(BUILD_DIR)/$(TARGET).bin
27
28 $(BUILD_DIR)/main.o: Src/main.c | $(BUILD_DIR)
29     arm-none-eabi-gcc -c $(CFLAGS) $< -o $@
30
31 $(BUILD_DIR)/startup_stm32l476xx.o: startup_stm32l476xx.s | $(BUILD_DIR)
32     arm-none-eabi-gcc -x assembler-with-cpp -c $(CFLAGS) $< -o $@
33
34 $(BUILD_DIR)/$(TARGET).elf: $(BUILD_DIR)/main.o $(BUILD_DIR)/startup_stm32l476xx.o
35     arm-none-eabi-gcc $^ $(LDFLAGS) -o $@
36
37 $(BUILD_DIR)/%.bin: $(BUILD_DIR)/%.elf | $(BUILD_DIR)
38     arm-none-eabi-objcopy -O binary -S $< $@
```

# Написать прошивку на расте

```
curl https://sh.rustup.rs -sSf | sh
```

```
rustup target add thumbv7em-none-eabihf
```

# Написать прошивку на расте

```
1      #![no_main]
2      #![no_std]
3
4       extern crate panic_halt;
5
6      use cortex_m::asm::bkpt;
7      use cortex_m_rt::entry;
8
9      #[entry]
10      fn main() → ! {
11         bkpt();
12         loop {}
13     }
```

# Написать прошивку на расте

```
1 rust_simple v0.1.0 (/project/rust_simple)
2   └─ cortex-m v0.6.0
3       └─ aligned v0.3.1
4           └─ as-slice v0.1.0
5               └─ generic-array v0.12.0
6                   └─ typenum v1.10.0
7                       └─ stable_deref_trait v1.1.1
8   └─ bare-metal v0.2.4
9       └─ [build-dependencies]
10          └─ rustc_version v0.2.3
11              └─ semver v0.9.0
12                  └─ semver-parser v0.7.0
13   └─ volatile-register v0.2.0
14       └─ vcell v0.1.0
15   └─ cortex-m-rt v0.6.7
16       └─ cortex-m-rt-macros v0.1.5
17           └─ proc-macro2 v0.4.27
18               └─ unicode-xid v0.1.0
19       └─ quote v0.6.11
20           └─ proc-macro2 v0.4.27 (*)
21       └─ rand v0.5.6
22           └─ rand_core v0.3.1
23               └─ rand_core v0.4.0
24       └─ syn v0.15.29
25           └─ proc-macro2 v0.4.27 (*)
26           └─ quote v0.6.11 (*)
27           └─ unicode-xid v0.1.0 (*)
28   └─ r0 v0.2.2
29   └─ panic-halt v0.2.0
```

# Написать прошивку на расте

```
19 [target.thumbv7em-none-eabihf]
20 runner = 'arm-none-eabi-gdb'
21 rustflags = [
22     "-C", "link-arg=-Tlink.x",
23 ]
-
1  /* Device specific memory layout */
2
3  /* This file is used to build the cortex-m-rt examples,
4     but not other applications using cortex-m-rt. */
5
6  MEMORY
7  {
8     /* FLASH and RAM are mandatory memory regions */
9     /* Update examples/data_overflow.rs if you change these sizes. */
10    FLASH : ORIGIN = 0x00000000, LENGTH = 256K
11    RAM : ORIGIN = 0x20000000, LENGTH = 64K
12
13    /* More memory regions can be declared: for example this is a second RAM region */
14    /* CCRAM : ORIGIN = 0x10000000, LENGTH = 8K */
15  }
16
17  /* The location of the stack can be overridden using the `_stack_start` symbol.
18     By default it will be placed at the end of the RAM region */
19  /* _stack_start = ORIGIN(CCRAM) + LENGTH(CCRAM); */
20
21  /* The location of the .text section can be overridden using the `_stext` symbol.
22     By default it will place after .vector_table */
23  /* _stext = ORIGIN(FLASH) + 0x40c; */
```

# Написать прошивку на расте

```
1      #![no_main]
2      #![no_std]
3
4      extern crate panic_halt;
5
6      use cortex_m::asm::{delay};
7      use cortex_m_rt::entry;
8      use stm32l4::stm32l4x6::{GPIOA, RCC};
9
10     #[entry]
11     fn main() → ! {
12         let rcc = unsafe { &*RCC::ptr() };
13         rcc.ahb2enr.modify(|_, w| w.gpioaen().set_bit());
14         rcc.ahb2rstr.modify(|_, w| w.gpioarst().set_bit());
15         rcc.ahb2rstr.modify(|_, w| w.gpioarst().clear_bit());
16
17         let gpioa = unsafe { &*GPIOA::ptr() };
18         gpioa.moder.modify(|_, w| w.moder5().output());
19         gpioa.otyper.modify(|_, w| w.ot5().push_pull());
20
21         loop {
22             gpioa.bsrr.write(|w| w.bs5().set_bit());
23             delay(1000000);
24             gpioa.bsrr.write(|w| w.br5().set_bit());
25             delay(1000000);
26         }
27     }
```

<https://github.com/rust-embedded/svd2rust>

# Написать прошивку на расте

```
1      #![no_main]
2      #![no_std]
3
4       extern crate panic_halt;
5
6      use cortex_m::asm::{bkpt, delay};
7      use cortex_m_rt::entry;
8
9      use stm32l4xx_hal::hal::digital::OutputPin;
10     use stm32l4xx_hal::gpio::GpioExt;
11     use stm32l4xx_hal::rcc::RccExt;
12
13     #[entry]
14      fn main() → ! {
15         let dp = stm32l4xx_hal::stm32::Peripherals::take().unwrap();
16         let mut rcc = dp.RCC.constrain();
17         let mut gpioa = dp.GPIOA.split(&mut rcc.ahb2);
18         let mut led = gpioa.pa5.into_push_pull_output(&mut gpioa.moder, &mut gpioa.otyper);
19
20         loop {
21             led.set_high();
22             delay(1000000);
23             led.set_low();
24             delay(1000000);
25         }
26     }
```

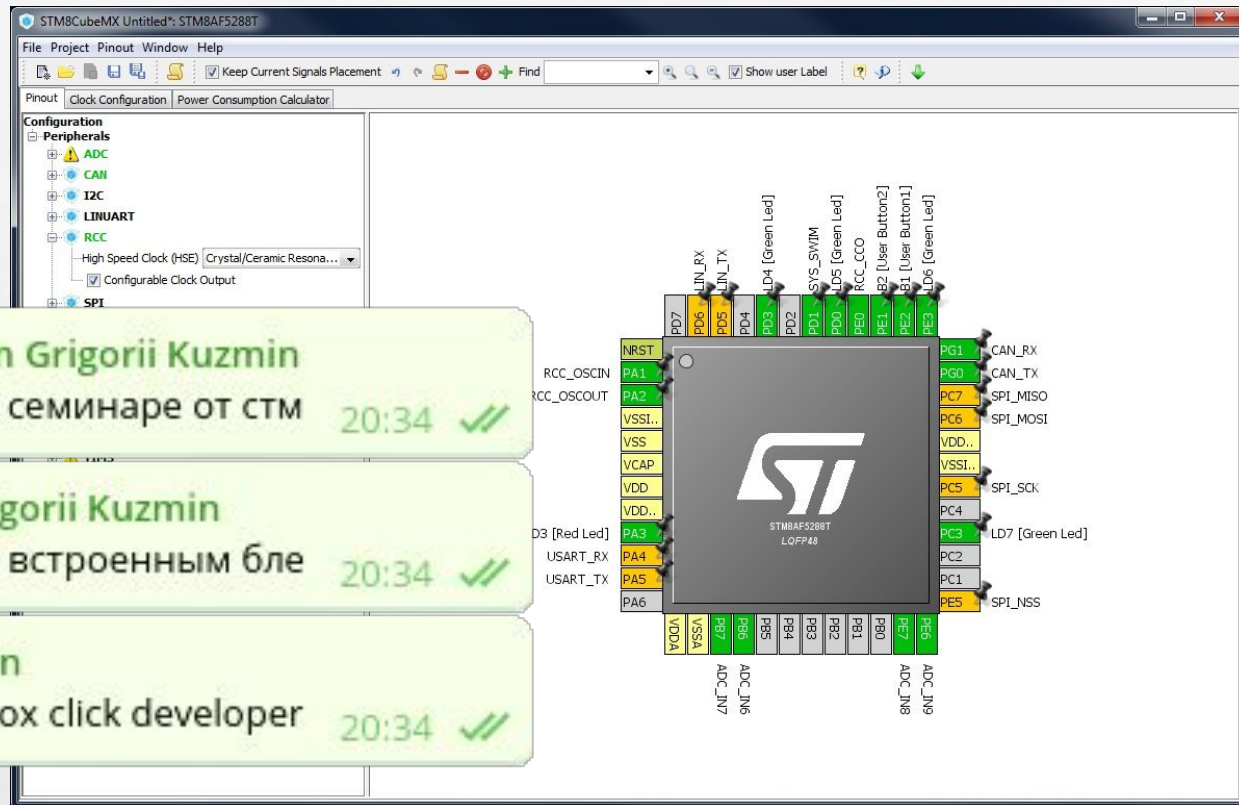
# Экосистема

- no\_std
- Jorge @japaric Aparicio
- Крейты из SVD
- embedded-hal и крейты
- cortex-m-rtfm
- heapless
- Всё остальное без аллокатора и std (serde, httparse, ...)

# Обзор SDK

- Куча кода написана вендорами
- Не все решения нормально стыкуются с растом
- Нужно интегрировать существующие решения

# Обзор SDK: STM32 HAL + CubeMX



# Обзор SDK: nRF SDK

```
3945  
3946 #ifndef NRFX_TWIM_NRF52_ANOMALY_109_WORKAROUND_ENABLED  
3947 #define NRFX_TWIM_NRF52_ANOMALY_109_WORKAROUND_ENABLED 0  
3948 #endif  
3949
```



**NORDIC**  
SEMICONDUCTOR

# Обзор SDK: nRF SDK

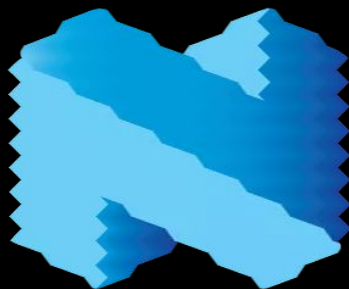
```
3945  
3946 #ifndef NRFX_TWIM_NRF52_ANOMALY_109_WORKAROUND_ENABLED  
3947 #define NRFX_TWIM_NRF52_ANOMALY_109_WORKAROUND_ENABLED 0  
3948 #endif  
3949
```



**NORDIC**  
SEMICONDUCTOR

# Обзор SDK: nRF SDK

```
3945  
3946 #ifndef NRFX_TWIM_NRF52_ANOMALY_109_WORKAROUND_ENABLED  
3947 #define NRFX_TWIM_NRF52_ANOMALY_109_WORKAROUND_ENABLED 0  
3948 #endif  
3949
```



# 0630p SDK: nRF SDK

```
3945  
3946 #ifndef NRFX_TWIM_NRF52_ANOMALY_109_WORKAROUND_ENABLED  
3947 #define NRFX_TWIM_NRF52_ANOMALY_109_WORKAROUND_ENABLED 0  
3948 #endif  
3949
```



# Обзор SDK: nRF SDK

```
3945
3946 #ifndef NRFX_TWIM_NRF52_ANOMALY_109_WORKAROUND_ENABLED
3947 #define NRFX_TWIM_NRF52_ANOMALY_109_WORKAROUND_ENABLED 0
3948 #endif
3949
```



**NORDI**  
SEMICONDUCTOR

# Обзор SDK

- Куча кода написана вендорами
- Не все решения нормально стыкуются с растом
- Нужно интегрировать существующие решения

# Обзор SDK: bindgen

```
372     .header("../src/cmsis_os.h")
373
374     .whitelist_type("osStatus")
375     .rustified_enum("osStatus")
376
377     .whitelist_type("osEvent")
378
379     .whitelist_function("osQueueCreate")
380     .whitelist_function("osQueueGet")
381     .whitelist_function("osQueuePut")
382
```

```
1256 osMessageQId osQueueCreate(size_t uxQueueLength, size_t uxItemSize) {
1257     return (osMessageQId)xQueueCreate(uxQueueLength, uxItemSize);
1258 }
```

```
513 extern "C" {
514     pub fn osQueueCreate(uxQueueLength: usize, uxItemSize: usize) -> osMessageQId
515 }
```

# Инфраструктура

- Докер
- Кроссборка (прошивка под линукс)
  - Обязка периферии
  - Обязка FreeRTOS
- Bitfield
- Asset + buildscript
- Static

# Работа с периферией. Static, mutex и все все все

```
static BLE_CTX: Static<os::Mutex<&mut bindings::nrf_ble_amts_t>> = Static::new();
```

```
BLE_CTX.init(os::Mutex::new(ble_ctx));
```

```
{  
    let ble_ctx = match BLE_CTX.get() {  
        Some(x) => x.lock(),  
        None => panic!("static uninitialized")  
    };  
  
    if ble_ctx.max_payload_len > 10 {  
        debug_println!("so much");  
    }  
}
```

```
let mut ble_ctx_p: Option<&mut bindings::nrf_ble_amts_t> = None;  
  
{  
    let ble_ctx = match BLE_CTX.get() {  
        Some(x) => x.lock(),  
        None => panic!("static uninitialized")  
    };  
  
    ble_ctx_p = Some(&mut ble_ctx);  
}
```

borrowed value does not live long enough

ой всё

<https://kviver.dev>

